

# PostgreSQL 与 MongoDB 处理非结构化数据性能比较

宗平<sup>1</sup>, 李雷<sup>2</sup>

ZONG Ping<sup>1</sup>, LI Lei<sup>2</sup>

1. 南京邮电大学 海外教育学院, 南京 210023

2. 南京邮电大学 计算机学院, 南京 210023

1.College of Overseas Education, Nanjing University of Posts and Telecommunications, Nanjing 210023, China

2.College of Computer, Nanjing University of Posts and Telecommunications, Nanjing 210023, China

**ZONG Ping, LI Lei. Performance comparison of PostgreSQL and MongoDB dealing with unstructured data. Computer Engineering and Applications, 2017, 53(7): 104-108.**

**Abstract:** Traditional relational databases can hardly store or handle massive unstructured data generated by Internet technology development efficiently. Varieties of NoSQL databases are capable to deal with unstructured data, but are poor in dealing with relational calculation under certain circumstances. Some new relational databases provide efficient ways to store both unstructured and structured data in multiple environments. In order to compare the data storage and processing capabilities of relational database and document-oriented NoSQL database quantitatively, this paper compares the unstructured data storage methods between hstore in PostgreSQL and embedded document in MongoDB. Performance and applicability analysis are also made by comparing performance of bulk loading, disk footprint, query on primary key and geometry coordinates.

**Key words:** PostgreSQL; hstore; MongoDB; embedded document

**摘 要:** 互联网技术的发展产生的海量非结构化数据在传统关系型数据库中难以被高速有效地进行存储和处理, 各类 NoSQL 数据库可以有效存储处理非结构化数据, 但是对关系运算功能的弱化难以满足应用场景的需求。具备非结构化数据处理能力的新型关系型数据库提供了适用多种应用场景的高效存储方式。为了能够定量地比较关系型数据库和面向文档的 NoSQL 数据库的数据存储与处理能力, 比较了 PostgreSQL 的 hstore 数据类型和 MongoDB 的内嵌文档对非结构化数据的储存方式, 并通过非结构化数据的批量加载、磁盘占用、主键查询、非主键查询、地理空间坐标查询等方面的对比来以分析性能特征与适用场景。

**关键词:** PostgreSQL 数据库; hstore 数据类型; MongoDB 数据库; 内嵌文档

**文献标志码:** A **中图分类号:** TP392 **doi:** 10.3778/j.issn.1002-8331.1508-0203

## 1 引言

随着互联网技术的发展, 如今的系统应用可以拥有几百万甚至更多的用户, 而且这些用户有可能是 7×24 小时、全年 365 天时刻接入某一应用。在这样的背景下, 如何选择能够支持海量用户连接的高并发数据库就显得尤为重要。

传统的关系型数据库基于关系数据模型。关系数据模型提供了一种简单而统一的模型, 以关联关系作为基本模型来描绘现实世界。所有的实体都被抽象为关

系, 实体间的附属属性和联系也被一同抽象成了关系。对于实体及其之间联系的一切操作也体现了相应的关联关系。为查询和管理结构化数据而设计的结构化查询语言 SQL (Structural Query Language), 可以在关系型数据库中对结构化数据进行包括新增 (create/insert)、查询 (select)、删除 (delete)、更新 (update) 在内的基本 CRUD 操作, 同时也可以进行高复杂度的连接 (join)、聚合 (group)、数据分区管理 (partition) 等操作<sup>[1]</sup>。因其易用性, SQL 推动了关系型数据库的发展和应用, 绝大多

**作者简介:** 宗平 (1956—), 男, 博士, 教授, 研究领域为网络技术、数据处理技术等, E-mail: zong@njupt.edu.cn; 李雷 (1987—), 男, 硕士研究生, 研究领域为分布式计算技术及其应用。

**收稿日期:** 2015-08-27 **修回日期:** 2015-10-20 **文章编号:** 1002-8331(2017)07-0104-05

**CNKI 网络优先出版:** 2015-10-28, <http://www.cnki.net/kcms/detail/11.2127.TP.20151028.1526.008.html>

数关系型数据库产品都把对SQL的支持作为数据库重要的一部分。

然而,随着各种应用对数据库要求的不断提高,尤其是互联网、多媒体和社交媒体等领域的新型应用对于高并发数据读写和海量非结构化数据结构处理需求的提出,传统的关系型数据库暴露出很多难以克服的问题<sup>[2]</sup>。磁盘IO的承受能力难以满足关系型数据库处理高并发读写数据请求;查询海量数据时关系型数据库效率及其低下;关系型数据库模式固定,难以进行横向扩展,带来较高维护成本和较低的可用性<sup>[3]</sup>。另外,在当今系统应用中,对于数据库一致性和读写实时性等需求明显降低,这些不再是必备需求的功能,其带来的高维护成本也难以让企业感到满意。NoSQL的出现很好地解决了上述关系型数据处理模式所面临的困境。

## 2 非关系型数据库

NoSQL(Not Only SQL)不是单一的一种工具,而是多种互补和互斥工具的集合。与关系型数据库不同,NoSQL数据库的主要优势在于它可以处理诸如文档、电子邮件、多媒体、社交媒体等非结构化数据<sup>[4]</sup>。非关系型数据库不使用RDBMS(Relational Data Base Management System)规则,模式不固定,数据模型简洁,对于存储不同类型的数据,非关系型数据库主要分为四大类<sup>[5]</sup>:

(1)键值数据库(Key-Value Database)。键值数据库可以被看作是拥有多行,但是仅由键和值两列组成的关系型数据库,数据以键值对的形式存储为一个阵列,通过这个唯一的键可以查找到与之关联的值,值的数据存储长度不受限制。这种基于键的查找值的方式可以大大缩短执行查询操作的时间,而且因为值可以是任何类型的数据,如对象、哈希表等,也可以极大地满足当今对非结构化数据模型灵活性的要求,非常适合模式经常发生改变的应用<sup>[6]</sup>。典型的键值数据库有Riak、Volde-mort和Redis等。

(2)列数据库(Column-Oriented Database)。列数据库主要是针对以下三种情况而设计的,即数量巨大的数据列、松散数据结构和多变的数据模式。在关系型数据库中数据行是被连续存储的,而在列数据库中每一列的值是连续存储的。这种存储设计上的变化使得诸如聚类查询、动态查询和即席查询等操作的性能有了极大的提高。大多数列数据库都适配了MapReduce框架,提高了将海量数据分布到多个系统上进行处理的速度<sup>[7]</sup>。目前比较受欢迎的开源列数据库有Hypertable、HBase和Cassandra。

(3)文档数据库(Document Database)。文档数据库是一类适合包含存储半结构化数据和执行动态查询

等功能的Web应用的NoSQL数据库,此类数据库将文档作为数据进行存储,文档以集合的形式组合在一起,并且文档都有唯一的键进行标识<sup>[8]</sup>。与关系型数据库相比,文档数据库中的集合(Collection)、文档(Document)就相当于表(Table)、记录(Record)。文档数据库是非常灵活的,不同的文档可以有不同的数据类型和结构的字段,在一个集合中新增文档时不会在同一集合中的其他文档中增加空白字段而造成空间浪费。目前,MongoDB、CouchDB和RavenDB是比较常用的文档数据库,其中MongoDB不仅能灵活处理面向文档的非结构化数据,提供复杂的数据查询方法,并且支持存储和处理空间地理坐标数据,提供类似于Oracle等关系型数据库中对空间地理坐标数据的查询和计算方法。本文将针对MongoDB存储处理非结构化数据进行研究。

(4)图形数据库(Graph Database)。此类数据库将数据库构建成网络结构,包含节点(Node)和代表节点之间关系的边(Edge)<sup>[9]</sup>。图形数据库可以快速地在大量数据之间查找关联关系,采用无索引的邻接操作代替了低效率的连接操作。几乎所有图形数据库都可以存储半结构化数据信息,不需要预先定义模式,易于进行模式改变和捕获即席数据关系。常见的图形数据库有Neo4j、Titan、OrientDB、AllegroGraph和InfiniteGraph等。

## 3 可处理非结构化数据的关系型数据库

虽然非关系型数据库的优越性能随着Web2.0技术的发展逐渐凸显,然而在很多情况下都会出现结构化数据和非结构化数据共存的场景。另外,由于关系型数据库体系多年的发展积累,其中包含的各种优秀的功能函数和框架是很多NoSQL数据库所不具备的,尤其是在某些应用场景下,非结构化数据的关系运算难以避免,而大多数NoSQL数据库都将关系型数据中强大的关系运算功能进行弱化,把关系运算操作提升至应用层进行处理,额外增加了应用开发成本和算法优化成本。因此,近年来一些优秀的开源关系型数据库开始寻求与非关系型数据库的结合,或是探索在关系型数据库中存储处理半结构化、非结构化数。比较具有代表性的是PostgreSQL数据库。

### 3.1 PostgreSQL数据库及hstore数据类型

PostgreSQL支持大多数基于最新ISO和ANSI标准的SQL数据库查询语句,保证数据库事务的ACID(Atomicity, Consistency, Isolation, Durability)。PostgreSQL的优势在于其在多平台上的适用性,构架的高度可扩展性,支持复杂数据类型以及成熟的代码库。PostgreSQL拥有三种存储非结构化数据的方法:XML、JSON和hstore<sup>[10]</sup>。由于支持GiST、GIN、B-Tree和Hsah等多种索引,hstore的速度明显高于XML和JSON。目前Post-

表1 hstore 部分常用操作

| 操作             | 描述                   | 举例                                           | 结果                         |
|----------------|----------------------|----------------------------------------------|----------------------------|
| hstore->text   | 获取一个 value           | 'a=>x,b=>y'::hstore->'a'                     | x                          |
| hstore->text[] | 获取多个 value           | 'a=>x,b=>y,c=>z'::hstore->ARRAY['c','a']     | { "z", "x" }               |
| hstore  hstore | 连接 hstore            | 'a=>b,c=>d '::hstore  'c=>x, d=>q'::hstore   | "a"=>"b","c"=>"x","d"=>"q" |
| hstore?text    | key 存在与否             | 'a=>1'::hstore?'a'                           | t                          |
| hstore?&text[] | 多个 key 存在与否          | 'a=>1,b=>2'::hstore?&ARRAY['a','b']          | t                          |
| hstore@>hstore | 左侧是否包含右侧             | 'a=>b,b=>1,c=>NULL'::hstore@>'b=>1'          | t                          |
| hstore<@hstore | 左侧是否包含右侧             | 'a=>c'::hstore<@'a=>b,b=>1,c=>NULL'          | f                          |
| hstore-text    | 删除一个 key             | 'a=>1,b=>2,c=>3'::hstore-'b'::text           | "a"=>"1","c"=>"3"          |
| hstore-text[]  | 删除多个 key             | 'a=>1,b=>2,c=>3'::hstore-ARRAY['a','b']      | "c"=>"3"                   |
| hstore-hstore  | 删除键值对                | 'a=>1,b=>2,c=>3'::hstore-'a=>4,b=>2'::hstore | "a"=>"1","c"=>"3"          |
| %%hstore       | 将 hstore 中的键值转换为数据   | %%'a=>foo,b=>bar'::hstore                    | {a,foo,b,bar}              |
| %#hstore       | 将 hstore 中的键值转换为二维数据 | %%#a=>foo,b=>bar'::hstore                    | {{a,foo},{b,bar}}          |

greSQL 的最新测试版本还推出了融合了 JSON 和 hstore 的 JSONB 类型,与 JSON 相比,JSONB 在 JSON 的基础上,由于采用了二进制的表示方法,其存储效率更高,索引速度更快。

PostgreSQL 的 hstore 数据类型可以在一个单独的字段中存储一个或多个键值对(Key-Value Pairs),通常用于存储元数据和半结构化数据,键和值都是文本字符串类型。hstore 的存储格式为 key=>value,其中 value 的值可以为空(NULL),但是 key 则不能为空。在同一个 hstore 字段中的 key 是唯一的,如果存入的 key 有重复,则仅能保存一个 value 值,但是不能确定保存的是其中的哪一个。hstore 部分常用操作<sup>[11]</sup>如表 1 所示,本文将针对使用广泛的 hstore 展开进一步研究。

### 3.2 hstore 与 MongoDB 内嵌文档存储比较

MongoDB 作为文档数据库的代表,支持类似于 JSON 的 BSON,数据结构十分松散,可以存储复杂数据结构和类型<sup>[12]</sup>。同时,MongoDB 还支持类似于面向对象的查询语言,可以实现关系型数据库中 SQL 的大部分功能,而且 MongoDB 还支持包含地理空间索引在内的多种索引。MongoDB 的众多特点之一是文档的嵌套存储,即在同一个字段中可以存储多个键值对,这一点与 PostgreSQL 的 hstore 十分类似。

与 PostgreSQL 的 hstore 相比,MongoDB 的内嵌文档是可以多层嵌套的,即内嵌文档中的字段里还可以继续存储内嵌文档,但是由于更新文档时总是将整个文档加载到内存,即使只更新部分内容也是如此,过多的在内嵌文档中再次嵌入文档会影响 MongoDB 的数据更新效率。hstore 不具有反复嵌套的功能,首先在数据结构的灵活性上已经明显低于 MongoDB;另外 hstore 的 key 和 value 都只能存储为文本字符串类型,而 MongoDB 的文档中可以存储包括布尔、32 位整数、64 位整数、64 位浮点数、字符串、日期、数组等多种数据类型<sup>[13]</sup>。表 2 和表 3 说明了分别以 hstore 和内嵌文档的方式存储于 PostgreSQL 和 MongoDB 两种数据库中的数据结构。

表2 hstore 数据存储方式(GEO 字段)

| ID     | NAME   | CITY      | ZIPCODE | GEO                                              |
|--------|--------|-----------|---------|--------------------------------------------------|
| 100011 | "NUPT" | "Nanjing" | 210023  | "type"=>"point","lon"=>"118.778","lat"=>"32.089" |

表3 内嵌文档数据存储方式

| 内嵌文档                        | 多层内嵌文档                      |
|-----------------------------|-----------------------------|
| {                           | {                           |
| {                           | "_id" : ObjectId("100011"), |
| "_id" : ObjectId("100011"), | "name" : "NUPT",            |
| "name" : "NUPT",            | "city" : "Nanjing",         |
| "city" : "Nanjing",         | "zipcode" : 210023,         |
| "zipcode" : 210023,         | "geo" : {                   |
| "geo" : {                   | "type" : "point",           |
| "type" : "point",           | "coordinates" : {           |
| "lon" : 118.778,            | "lon" : 118.778,            |
| "lat" : 32.089              | "lat" : 32.089              |
| }                           | }                           |
| }                           | }                           |
| }                           | }                           |

## 4 性能对比测试

近年来基于地理位置的服务(LBS)和相关应用越来越多,电信运营商和城市管理部门对于设备资源位置的数字化管理需求增高,对设备资源非结构化的供应商信息和设备资源本身的位置信息进行有效的存储和查询成为 LBS 提供方需要解决的首要问题之一。

本文使用 PostgreSQL 和 MongoDB 处理带有非结构化属性的地理数据,从多方面进行性能测试,对比分析其适用场景。实验数据采用的是表 4 所示的文档结构。其中,GEO 字段为地理空间坐标字段,关系型数据库模型中以 PostGis 格式存储<sup>[14]</sup>,hstore 中以 WKT 文本格式存储,MongoDB 中以 GeoJSON 内嵌文档格式存储<sup>[15]</sup>;ID 为 64 位整数,但在 hstore 中全部为字符串类型;SCHOOL 和 ADDR 字段可有可无,在关系型数据库模型中存入空值(NULL);其余字段均为字符串类型;取用 180 万条数据,每条数据平均 63 Byte。



表4 实验数据的文档结构

| 序号 | 字段       |
|----|----------|
| 1  | ID       |
| 2  | NAME     |
| 3  | SCHOOL   |
| 4  | GEO      |
| 5  | ADDR     |
| 6  | CITY     |
| 7  | PROVINCE |
| 8  | ZIPCODE  |

为了对比 PostgreSQL 传统关系数据模型、hstore 格式和非关系型数据库 MongoDB 的性能差异,对比使用不同类型的索引时对数据存储和查询效率的影响,保证性能测试的全面性。本文采用以下五种主要影响性能的因素进行了对比测试分析。

- (1)RDB:主键使用 B-Tree 索引的 PostgreSQL 传统关系数据库模式。
- (2)HSTORE:采用无索引 hstore 字段的 PostgreSQL 关系数据库。
- (3)GiST:使用 hstore 加 GiST 索引的 PostgreSQL 关系数据库。
- (4)GIN:使用 hstore 加 GIN 索引的 PostgreSQL 关系数据库。
- (5)MONGODB:主键使用 B-Tree 索引的 MongoDB 非关系型数据库。

在对以上模型进行数据插入之前,首先对其建立相应的索引。

实验用计算机操作系统为 64 位 Windows 7,CPU 为 Intel Core i7 2.8 GHz,硬盘 7200 RPM,内存 8 GB;测试用数据库版本为 PostgreSQL9.3 和 MongoDB3.0.5。

4.1 批量加载测试

通过读取含有 180 万条数据的 SVC 文件,将其解析成为合适的格式,之后插入数据库,对比所耗用的时间,其中包括解析花去的时间。

从图 1 中的测试结果中可以看出,传统关系数据库模式批量加载速度最快;使用 B-Tree 索引的 MongoDB 略慢于不使用索引的 hstore 存储方式;而在 hstore 上使用 GiST 索引和 GIN 索引则耗费了大量的处理时间,数据批量加载速度最慢。因此,当经常出现数据批量加载的情况时,不建议在如 GiST 和 GIN 方式中为所有数据

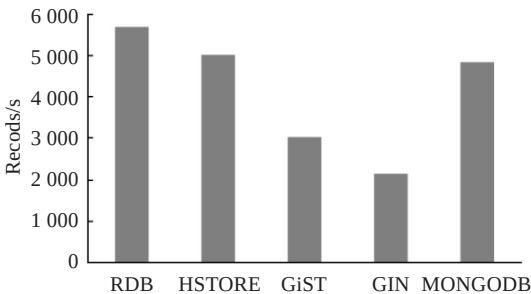


图1 批量加载测试结果对比

字段添加索引,而应使用无索引或仅有主键索引的存储方式(如 RDB、HSTORE、MONGODB 方式)。另外,在要求采用类似 GiST 和 GIN 存储方式的特殊情况下,建议在数据批量加载之前删除原有索引,待数据加载完毕后对索引进行重建。

4.2 磁盘占用测试

数据批量加载完毕后,可以进行磁盘占用测试。PostgreSQL 数据库采用 pg\_\*.size 方法获取资源的磁盘占用情况;MongoDB 通过 db.stats() 命令来获取资源的磁盘占用情况。磁盘占用测试结果如图 2 所示。

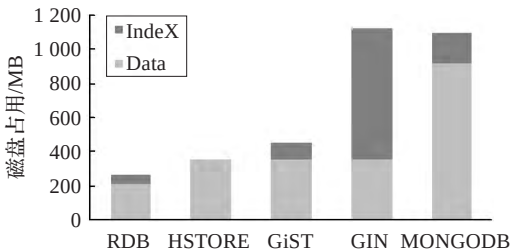
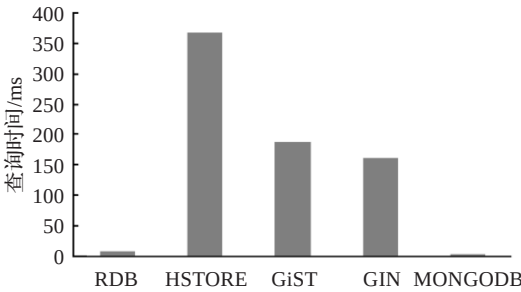


图2 磁盘占用测试结果对比

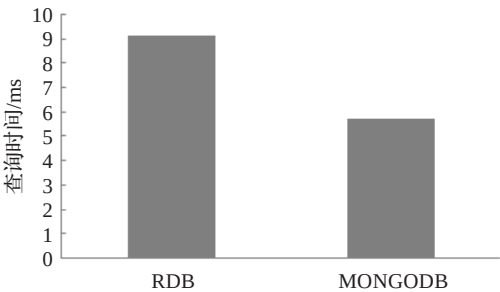
从五种存储方式占用磁盘空间的总量以及其中数据和索引的占比来看,PostgreSQL 存储数据的效率明显高于 MongoDB,而 hstore 的 GIN 索引却严重耗费了大量的磁盘空间。在磁盘资源紧张,且数据类型多为字符串的场景中,对全部采用 hstore 格式的数据建议使用磁盘利用率更高的 GiST 索引。MongoDB 的预分配机制会使数据文件大小以指数级别增长,但是通过下一项测试可以看出,这是在用空间换取时间。

4.3 主键查询测试

随机选取 100 条样本数据,对其中一条数据的主键字段 ID 进行查询操作,对比五种存储方式的查询时间,如图 3 所示。PostgreSQL 的查询时间从 Python 的



(a) 五种方式主键查询测试结果对比



(b) 主键查询测试结果对比 (<10 ms)

图3 主键查询测试结果对比

execute()方法获取,MongoDB的查询时间从fetch()方法获取。

对比结果可以得知,对于hstore自身来说,GiST和GIN索引带来的查询速度提升一倍以上,而B-Tree索引的效率是GiST和GIN的20倍以上。对于非结构化数据量占比较大且对关系运算要求不高的应用,推荐使用MongoDB;若使用hstore格式存储数据,则最好能建立空间占用相对较少而查询速度相对较快的GiST索引。

#### 4.4 非主键查询测试

RDB和MONGODB两种存储方式中只对主键字段ID建立了B-Tree索引,HSTORE无索引,因此在查询非主键时,这三种存储方式将会进行全表扫描操作。生成100条数据的NAME样本数据集,分别在五种存储方式中对这100条数据进行查询,对比耗时,获取查询时间方法与测试三相同。

由于GiST索引和GIN索引是对整个hstore有效,而不仅仅是主键字段,因此这两种存储方式的非主键查询速度优于其他三种非主键上无索引的存储方式;在查询条件较多且不确定的场景中,建议使用GiST或GIN方式,同时也可根据实际应用需求,对RDB和MONGODB方式的常用查询字段添加适当的索引以提高查询速度。非主键查询测试结果如图4所示。

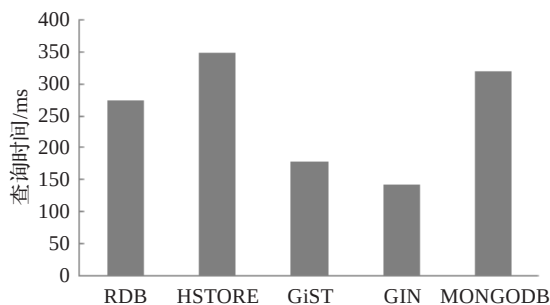


图4 非主键查询测试结果对比

#### 4.5 地理空间坐标查询测试

在RDB的GEO字段上建立GiST索引,在MONGODB的GEO字段上建立2dsphere索引,其余数据结构和索引不变。选定一块坐标区域,查询坐标位于此区域内的所有记录,RDB采用PostGis的ST\_Within()方法查询,MONGODB采用\$geoWithin方法查询,其余存储方式采用坐标范围比对的方法进行查询,如图5所示。查询结果为1844条记录。

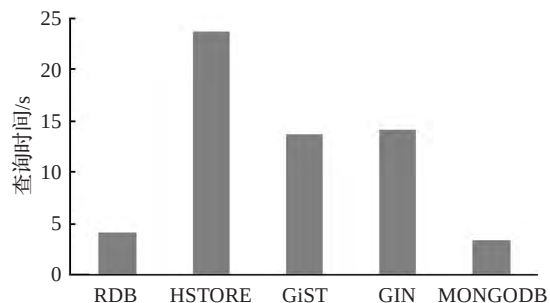


图5 地理空间坐标查询测试结果对比

从结果可以明显看出,带有专门地理空间数据结构和索引的RDB和MONGODB对于坐标计算查询耗时最少,而且MONGODB略胜一筹;而无地理空间模型支持的HTSTORE、GiST和GIN进行坐标查询计算时明显吃力,耗时已经超过了大多数LBS应用的承受范围。MongoDB自带的地理空间方法目前还比不上PostGis,而目前LBS采集的数据除了坐标之外的其他数据模式不固定,因此,除非是要进行高复杂度的坐标关系计算,MongoDB可作为LBS应用开发的首选。

#### 5 结束语

从关系型数据库转换为非关系型数据库面临着多方面的挑战,比如对各种类型非关系型数据库的深入研究,探寻特定场景下的最佳解决方案,以及开发可以实现与关系型数据库完全一样的功能和查询操作的非关系型数据库。未来关系型数据库仍会被继续沿用,而开发者所要做的是根据应用需求来选择合适的数据库。本文从多方面对MongoDB和PostgreSQL在处理非结构化数据方面的性能进行了比对分析,虽然MongoDB在磁盘占用方面不如PostgreSQL高效,但是MongoDB对多种数据类型的支持以及高效的索引机制带来的快速查询在非关系型数据库中是有明显优势的。对于无模式的非结构化数据,使用支持包括地理空间索引在内的MongoDB可以使系统应用同时获得强大的功能和较高的查询速度。但是,MongoDB目前还需要靠文档引用和内嵌文档的方式替代关系型数据库中复杂的连接操作,对于带有部分简单非结构化数据且需要进行复杂关系运算操作的应用,建议使用关系数据模型加hstore数据类型结合的方法进行存储。

#### 参考文献

- [1] ISO/IEC 9075-1: 1999 Information technology—Database languages—SQL[S]. International Organization for Standardization, 1999.
- [2] Gudivada V N, Rao D, Raghavan V V. Nosql systems for big data management[C]//Proceedings of IEEE World Congress on Services (SERVICES), 2014: 190-197.
- [3] Banker K. MongoDB in action[M]. [S.l.]: Manning Publications Co, 2011.
- [4] Zhu Weiping, Li Mingxin, Chen Huan. Using MongoDB to implement textbook management system instead of MySQL[C]//Proceedings of IEEE 3rd International Conference on Communication Software and Networks (ICCSN), 2011: 303-305.
- [5] Gyorodi C, Gyorodi R, Pecherle G, et al. A comparative study: MongoDB vs. MySQL[C]//Proceedings of the 13th International Conference on Engineering of Modern Electric Systems (EMES), 2015: 1-6.

(下转 170 页)

- 
- [8] Xie S, Shan S, Chen X, et al. Fusing local pattern of Gabor magnitude and phase for face recognition[J]. IEEE Transactions on Image Processing, 2010, 19(5): 1349-1361.
  - [9] Heikkilä M, Pietikäinen M, Schmid C. Description of interest regions with center-symmetric local binary patterns[J]. Lecture Notes in Computer Science, 2006, 42(3): 58-69.
  - [10] 王宪, 张彦, 慕鑫, 等. 基于改进的LBP人脸识别算法[J]. 光电工程, 2012, 39(7): 109-114.
  - [11] 杨恢先, 翟云龙, 蔡勇勇, 等. NSST与CSLDP相结合的人脸识别[J]. 计算机工程与应用, 2016, 52(8): 135-140.
  - [12] 盛光磊, 张腾, 裴铮. 基于3×3块梯度图像划分和人脸几何模型的人脸检测[J]. 电脑开发与应用, 2013(1): 74-75.
  - [13] Zhang T, Tang Y Y, Fang B, et al. Face recognition under varying illumination using gradient faces[J]. IEEE Transactions on Image Processing a Publication of the IEEE Signal Processing Society, 2009, 18(11): 2599-2606.
  - [14] Chen X, Zhang J. Illumination Robust single sample face recognition using multi-directional Orthogonal Gradient Phase faces[J]. Neurocomputing, 2011, 74: 2291-2298.
  - [15] 杨恢先, 蔡勇勇, 翟云龙, 等. 基于正交梯度二值模式的单样本人脸识别[J]. 计算机应用, 2014, 34(2): 546-549.
  - [16] Wang B, Li W, Yang W, et al. Illumination Normalization based on Weber's law with application to face recognition[J]. IEEE Signal Processing Letters, 2011, 18(8): 462-465.
- 

(上接96页)

- [8] Zha Z J, Mei T, Wang J, et al. Graph-based semi-supervised learning with multiple labels[J]. Journal of Visual Communication & Image Representation, 2009, 20(2): 97-103.
  - [9] Tao J, Wen S, Hu W.  $L_1$ -norm locally linear representation regularization multi-source adaptation learning[J]. Neural Networks, 2015, 69: 80-98.
  - [10] Nie F, Xu D, Tsang W H, et al. Flexible manifold embedding: A framework for semi-supervised and unsupervised dimension reduction[J]. IEEE Transactions on Image Processing, 2010, 19(7): 1921-1932.
  - [11] Golub G H, Loan C F. Matrix computations[M]. 3rd ed. Baltimore, MD: Johns Hopkins Univ Press, 1996.
  - [12] Gretton A, Fukumizu K, Harchaoui Z, et al. A fast, consistent kernel two-sample test[C]//Advances in Neural Information Processing Systems, Vancouver, British Columbia, Canada. 2009: 673-681.
  - [13] Yang J, Yan R, Hauptmann A G. Cross-domain video concept detection using adaptive SVMs[C]//Proceedings of International Conference on Multimedia, Augsburg, Germany, 2007: 188-197.
  - [14] Han Y, Wu F, Zhuang Y, et al. Multi-label transfer learning with sparse representation[J]. IEEE Transactions on Circuits & Systems for Video Technology, 2010, 20(8): 1110-1121.
  - [15] Duan L, Xu D, Tsang W H. Domain adaptation from multiple sources: A domain-dependent regularization approach[J]. IEEE Transactions on Neural Networks & Learning Systems, 2012, 23(3): 504-518.
- 

(上接108页)

- [6] Nyati S S, Pawar S, Ingle R. Performance evaluation of unstructured NoSQL data over distributed framework[C]//Proceedings of International Conference on Advances in Computing, Communications and Informatics (ICACCI), 2013: 1623-1627.
- [7] Dean J, Ghemawat S. MapReduce: Simplified data processing on large clusters[J]. Communications of the ACM, 2008, 51(1): 107-113.
- [8] Zhang S. Application of document-oriented NoSQL database technology in Web-based software project documents management system[C]//Proceedings of 2013 International Conference on Information Science and Technology (ICIST), 2013: 504-507.
- [9] Kaur K, Rani R. Modeling and querying data in NoSQL databases[C]//Proceedings of IEEE International Conference on Big Data, 2013: 1-7.
- [10] Obe R O, Hsu L S. PostgreSQL: Up and running: A practical introduction to the advanced open source database[M]. [S.l.]: O'Reilly Media, Inc, 2014.
- [11] PostgreSQL. PostgreSQL 9.3.9 documentation[EB/OL]. [2015-08-02]. <http://www.postgresql.org/docs/9.3/static/index.html>.
- [12] Zhao G, Huang W, Liang S, et al. Modeling MongoDB with relational model[C]//Proceedings of the 4th International Conference on Emerging Intelligent Data and Web Technologies (EIDWT), 2013: 115-121.
- [13] MongoDB. MongoDB manual[EB/OL]. [2015-08-02]. <http://docs.mongodb.org/manual/>.
- [14] Obe R, Hsu L. PostGIS in action[M]. [S.l.]: Manning Publications Co, 2011.
- [15] Butler H, Daly M, Doyle A, et al. The GeoJSON format specification, Rapport Technique[R]. 2008.